

COMS7071A Assignment: Grid2Op using DQN and PPO

Willem Van Der Merwe 2914429



University of the Witwatersrand, Johannesburg

25 October 2024

Chapter 1

Model Architectures

The reinforcement learning algorithms selected for this study was Proximal Policy Optimization (PPO) [8] and Deep Q-Network (DQN) [5]. These algorithms were implemented and iteratively improved to optimize power grid operations in the Grid2Op environment spesifically the 'l2rpn_case14_sandbox' variant. All artifacts can be found at the GitHub: https://github.com/wjvandermerwe/RL_Assignment.git

1.1 Deep Q-Network (DQN)

DQN [5] is a value-based method that approximates the optimal action-value function using deep neural networks, which is suitable for handling high-dimensional state spaces. DQN relies on a Q-learning framework combined with a deep neural network to approximate the optimal Q-values for each state-action pair, which enables the agent to make decisions that maximize expected rewards. Due to this algorithm already being part of prior assessments I don't cover the architecture in-detail.

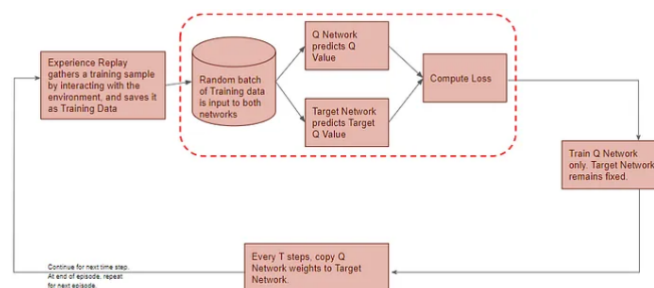


FIGURE 1.1: DQN Architecture [2]

1.2 Proximal Policy Optimization (PPO)

PPO [8] is a policy gradient method designed to optimize a parameterized policy by balancing exploration and exploitation while avoiding large, destabilizing policy updates. PPO's stability and efficiency stem from its use of a clipped surrogate objective function to limit the extent of each policy update. It belongs to the category of policy optimization methods and utilizes an Actor-Critic architecture, which involves updating both the policy (actor) and the value function (critic). The primary goal of PPO is to constrain policy updates to prevent large steps that can destabilize learning. Unlike traditional policy gradient methods, PPO uses a clipped surrogate objective to regulate policy updates, thus ensuring stable and reliable training. Below, the structure of PPO is detailed:

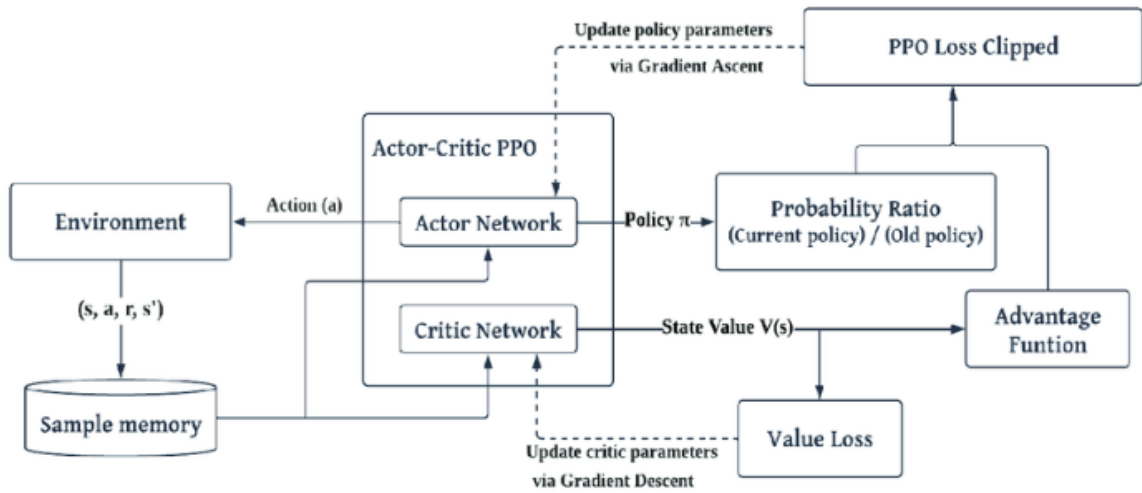


FIGURE 1.2: PPO Architecture [4]

Actor-Critic Architecture: PPO uses an actor network, which outputs a probability distribution over actions, and a critic network, which estimates the value function that gives the expected return from a given state. The actor explores the action space by selecting actions probabilistically, while the critic provides feedback on the quality of the selected actions, thereby guiding the actor's learning.

Clipped Surrogate Objective: PPO maintains stability through the use of a clipped surrogate objective function. The objective function for PPO is given by [8]:

$$L^{CLIP}(\theta) = \mathbb{E}_t [\min (r_t(\theta)\hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon)\hat{A}_t)]$$

Where,

$$r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)}$$

represents the ratio of the probability of selecting action a_t under the updated policy to the probability under the old policy, and $\hat{A}_t$ represents the advantage function. The advantage function quantifies how much better the chosen action is compared to the average action for the given state. The clipping operation constrains $r_t(\theta)$ to lie within the interval $[1 - \epsilon, 1 + \epsilon]$, preventing updates that are excessively large or too small.

Advantage Estimation: PPO employs Generalized Advantage Estimation (GAE) [8] to compute the advantage function $\hat{A}_t$. GAE is useful for reducing the variance in policy gradient estimates while preserving their accuracy, leading to more stable and efficient learning.

$$\hat{A}_t = \sum_{l=0}^{\infty} (\gamma\lambda)^l \delta_{t+l}$$

Where γ is the discount factor, and λ controls the bias-variance tradeoff. The temporal difference (TD) error δ_t measures how well the value function estimates the return compared to the observed outcome.

Entropy Regularization: Entropy regularization is employed in PPO to ensure that the policy maintains adequate exploration [6] throughout the learning process. By adding an entropy term to the objective function, PPO encourages the policy to remain diverse in its action selection, and therefore prevent premature convergence to suboptimal strategies.

Policy Updates: The policy is updated using stochastic gradient ascent, aiming to maximize the clipped surrogate objective. To ensure stable and gradual learning, PPO performs multiple epochs of minibatch updates with a small learning rate.

Implementation Details: During training, PPO switches between collecting data by running the current policy in the environment and optimizing the clipped surrogate objective using the collected data. Key hyperparameters include the clipping threshold ϵ , the discount factor γ , the GAE parameter λ , and the number of epochs per iteration [6].

Chapter 2

Model Iteration Implementations

The approach was to apply theoretical improvements from prior research to enhance both core models directly. However, specifically for PPO, it was not possible to combine all proposed changes in sequence due to conflicting modifications. The methodology employed an iterative improvement process for both PPO and DQN, beginning with a base implementation of each algorithm. Three successive iterations of enhancements were then applied to each agent. Each improvement was intended to incrementally boost agent performance and robustness, providing an in-depth analysis of the impact of each enhancement.

Chained improvements are noted at the bottom for ease of reference.

2.1 DQN Improvements

2.1.1 Iteration 1: Double DQN (2DQN)

2DQN [3] addresses the fundamental limitation of DQN, namely overestimation bias in action-value predictions. In traditional DQN, the same network is used for both action selection and evaluation, which tends to result in overly optimistic value estimates. Double DQN resolves this by using two separate networks—one to select actions and the other to evaluate their value.

The key difference lies in how the target values are calculated. Instead of using the maximum Q-value estimated by the same network, 2DQN uses the main network to select the action and the target network to evaluate its value [3]. This decoupling of action selection from action evaluation reduces the upward bias and results in more accurate value estimates.

The implementation of 2DQN involves maintaining two neural networks: the primary Q-network and the target Q-network. The target network is updated periodically, providing a stable reference point for value calculations, while the main network is updated at each step. This helps maintain stability in the learning process by preventing rapid shifts in target estimates.

Main network,

$$a'_{max} = \arg \max_a Q_{main}(s', a')$$

target network

$$Q(s, a) = r + \gamma Q_{target}(s', a'_{max})$$

This split reduces the likelihood of overestimation by breaking the dependency between action selection and Q-value estimation in the same network [3], leading to more accurate and stable target values.

from the results [3] the runtime changes expected to be seen for 2DQN, are improved stability, policy learning should also be more reliable thus more accurate action values, and lastly better convergence could be expected.

Note: This is chained with the base DQN implementation.

2.1.2 Iteration 2: Dueling DQN (DDQN)

DDQN [10] introduces a change that separates the estimation of the state-value function $V(s)$ from the advantage function $A(s, a)$. The state-value function estimates the value of being in a particular state, while the advantage function estimates the relative benefit of each possible action compared to an average action in that state.

The key concept is that not all actions are equally important in every state. In many situations, the value of a state is independent of the specific action taken. By having separate streams for state-value and advantage estimation, Dueling DQN allows the network to focus on learning the value of the state itself without being overly

influenced by the choice of action.

$$Q(s, a) = V(s) + (A(s, a) - \frac{1}{|A|} \sum_{a'} A(s, a'))$$

where,

$$A(s, a) - \frac{1}{|A|} \sum_{a'} A(s, a')$$

normalizes the advantage function to ensure the state-value $V(s)$ represents the state independently of action biases.

From [10] it is indicated that the implementation should be more efficient learning process with faster convergence. The separation of value and advantage leads to improved stability, particularly in environments where many actions yield similar outcomes.

Note: This is chained onto the 2DQN implementation.

2.1.3 Iteration 3: Prioritized Experience Replay (PER)

PER [7] enhances the efficiency of replay buffer, by assigning a priority value to each transition in the replay buffer, based on its temporal difference (TD) error.

$$\text{TD Error : } \delta = |Q(s, a) - (r + \gamma \max_{a'} Q(s', a'))|$$

Transitions with larger TD errors indicate greater learning potential, as they represent situations where the agent's prediction was significantly incorrect.

The core idea behind PER is to focus the learning process on the experiences that will have the greatest impact on improving the policy. Priority assignment is done via selection of transitions with the higher TD error, and importance sampling for balancing the priorities.

Implementation of PER requires modifying the replay buffer to maintain priority values for each transition. During sampling, transitions with higher priority are

more likely to be replayed, ensuring that the agent focuses on learning from its most informative experiences. Additionally, importance sampling weights are used to correct any bias introduced by the prioritized sampling process, ensuring stable updates.

From [7] the implementation should accelerate the convergence of the learning process. further more expected increased learning efficiency and policy update stability should be consequences.

Note: This is chained onto the DDQN implementation

2.2 PPO Improvements

2.2.1 Iteration 1: Truly PPO

The "Truly PPO" version proposed by [9] aims to address limitations of the original PPO by enhancing the adaptability of the clipping mechanism. The clipping threshold is dynamic rather than fixed, enabling policy updates that are more context-sensitive. This adaptability results in more reliable learning, particularly in environments with diverse state distributions.

standard PPO,

$$L^{CLIP}(\theta) = E[\min(r(\theta)\dot{A}, \text{clip}(r(\theta), 1 - \varepsilon, 1 + \varepsilon)\dot{A})]$$

truly PPO modifies this by updating the ε term with a few hyperparameters:

$$\varepsilon = f(\text{trainphase}, \text{rewardstructure}, \text{statediversity})$$

Implementing "Truly PPO" necessitates careful hyperparameter adjustment to ensure optimal clipping behavior, which introduces computational overhead but yields

significant improvements in policy stability and convergence. Higher reward fluctuations may prompt a tighter clip region. Early phases may allow broader exploration, with more restrictive clipping in later stages. More diverse states may benefit from broader updates to better explore complex environments

Note: This is chained onto the base PPO implementation.

2.2.2 Iteration 2: Intrinsic Curiosity Module (ICM)

The Intrinsic Curiosity Module augments the standard reward signal with an intrinsic reward based on prediction error. The ICM consists of a forward model and an inverse model. The forward model predicts the next state given the current state and action, while the inverse model predicts the action given consecutive states. The magnitude of the prediction error from the forward model serves as the intrinsic reward, which encourages the agent to visit states where it has a high level of uncertainty.

The intrinsic reward given by,

$$r_t^{int} = \frac{1}{2} ||s_{t+1} - \hat{s}_{t+1}||^2$$

The Inverse Model Loss,

$$\text{Loss}^{inv} = - \sum_i a_{t,i} \log(\hat{a}_{t,i})$$

Total Loss,

$$\text{Total Loss} = (1 - \beta) \cdot \text{Loss}^{inv} + \beta \cdot \text{Loss}^{fwd}$$

The primary goal of ICM is to motivate exploration, particularly in environments where external rewards are sparse or delayed. In power grid operations, ICM might encourage the agent to explore a wider range of control actions, which could lead to more robust grid management strategies.

From an implementation perspective, integrating ICM requires modifying the neural network architecture to include the forward and inverse models, increasing computational complexity and training requirements. However, this added complexity

is justified by the resulting improvements in exploration efficiency, leading to more robust policy performance.

Note: The Implementation of ICM did not converge.

2.2.3 Iteration 3: Self Imitation Learning (SIL)

SIL enhances an agent's learning process by enabling it to self-learn from past successful experiences. The key idea is to have the agent revisit and imitate actions that led to high returns, reinforcing behaviors that proved beneficial, even in the absence of external rewards. This is particularly valuable for environments with sparse or delayed rewards, as it allows the agent to learn from its own "demonstrations" of successful strategies.

SIL Reinforces High-Reward Actions meaning it encourages actions that lead to cumulative returns higher than the current value estimate. This creates a secondary learning objective, focusing on leveraging the agents best experiences. It also Imitates Successful Trajectories, By revisiting successful experiences, SIL helps the agent improve policy robustness and efficiency, particularly in complex environments.

The components for SIL include:

Advantage Computation

$$\text{Advantage} = \max(R - V(s), \text{threshold})$$

where R is the cumulative return from past experiences, and $V(s)$ is the current value estimate for the state s . This ensures only positive advantages are used.

SIL Policy Loss

$$L_{\text{SIL policy}} = -\log \pi(a|s) \text{Advantage}$$

where $\pi(a|s)$ is the probability of taking action a in state s under the current policy.

SIL Value Loss

$$L_{SILValue} = \max(R - V(s), 0)^2$$

This squared error penalizes underestimation of high-return states, helping the agent prioritize learning from successful trajectories.

Combined PPO and SIL

$$\text{Total Loss} = L^{PPO} + \alpha(L_{SILpolicy} + c \cdot L_{SILValue})$$

Chapter 3

Runtime And Evaluations

3.1 Environment Setup

The Grid2Op environment was used to simulate power grid operations. Specifically, the "l2rpn_case14_sandbox" environment is chosen for its simplicity and focus on power flow management without adversarial components. The observation and action spaces will be adapted to suit both PPO and DQN agents, ensuring that the agents can effectively interact with the environment. The reward function used in this study is a **CombinedScaledReward**, which is a weighted combination of the L2RPN Reward and the N-1 Reward. This combined reward encourages the agents to maximize power flow while maintaining grid resilience by ensuring there are no single points of failure.

3.2 Training

3.2.1 DQN and Iterations

3.2.2 PPO and Iterations

The models were iteratively trained using different step sizes to gain insights into the various stages of training. Notably, early convergence issues arose, where the models initially converged but later diverged beyond a certain step size, possibly indicating overfitting.

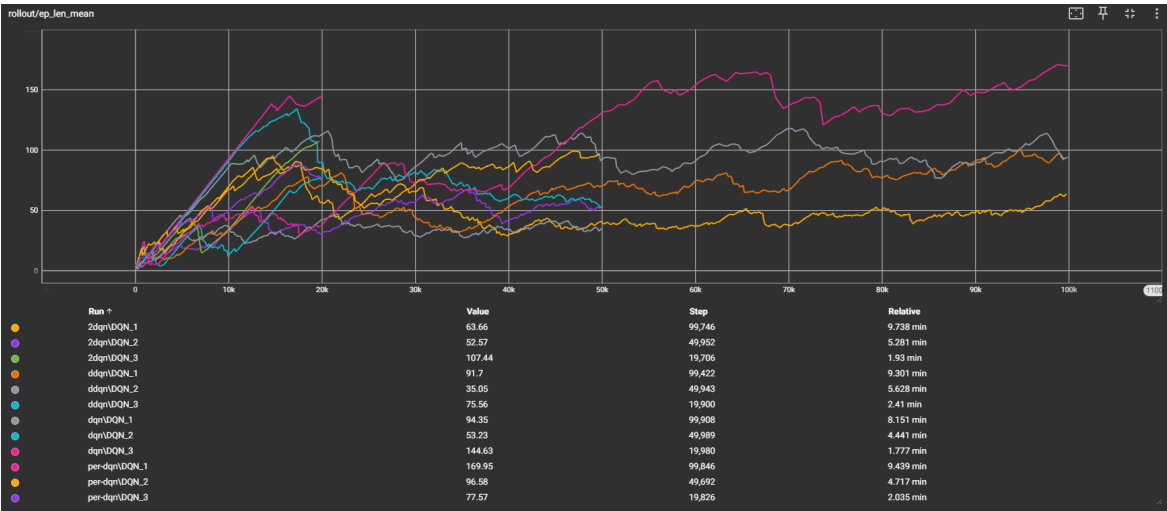


FIGURE 3.1: DQN episode lengths over training steps

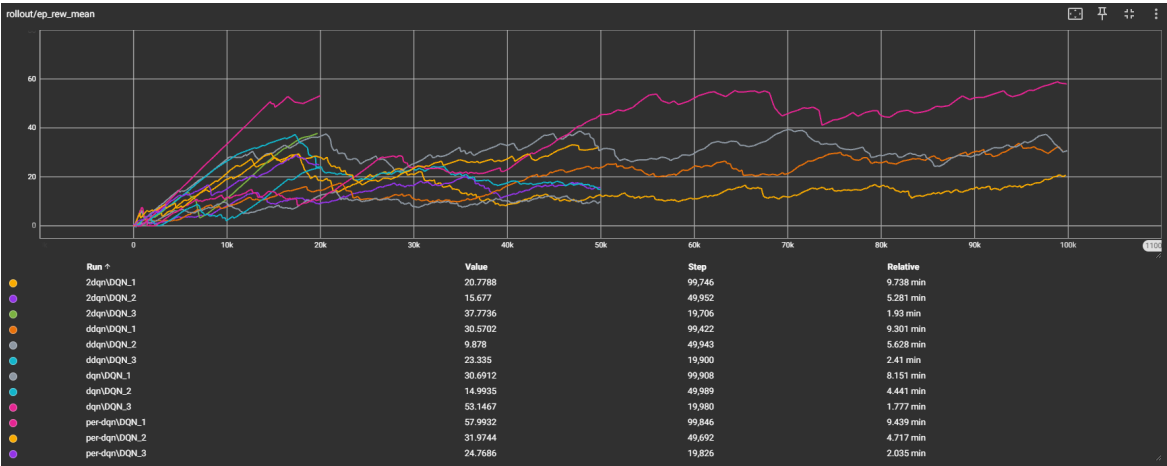


FIGURE 3.2: DQN cumalitive rewards over training steps

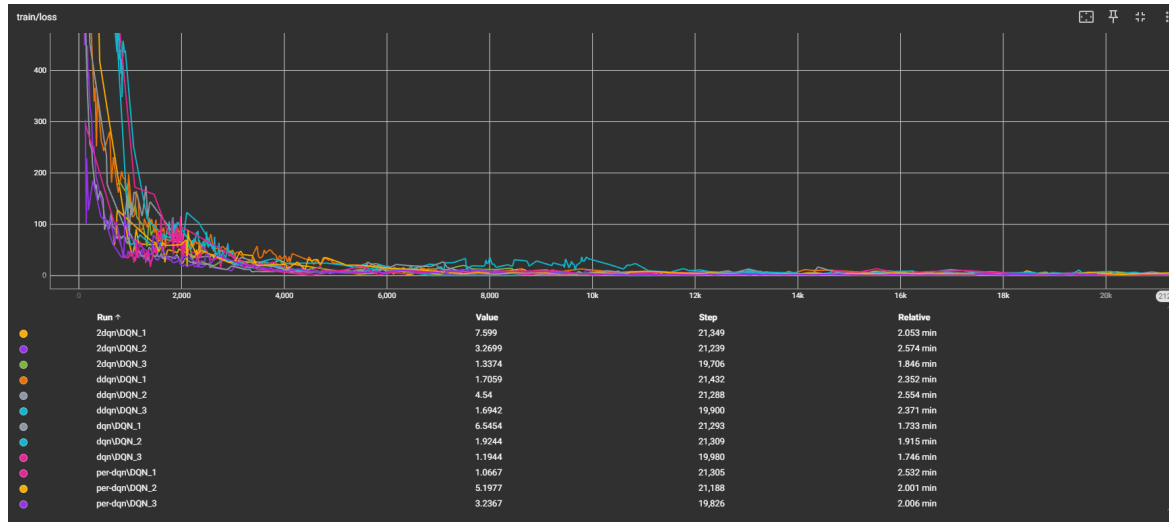


FIGURE 3.3: DQN loss over training steps

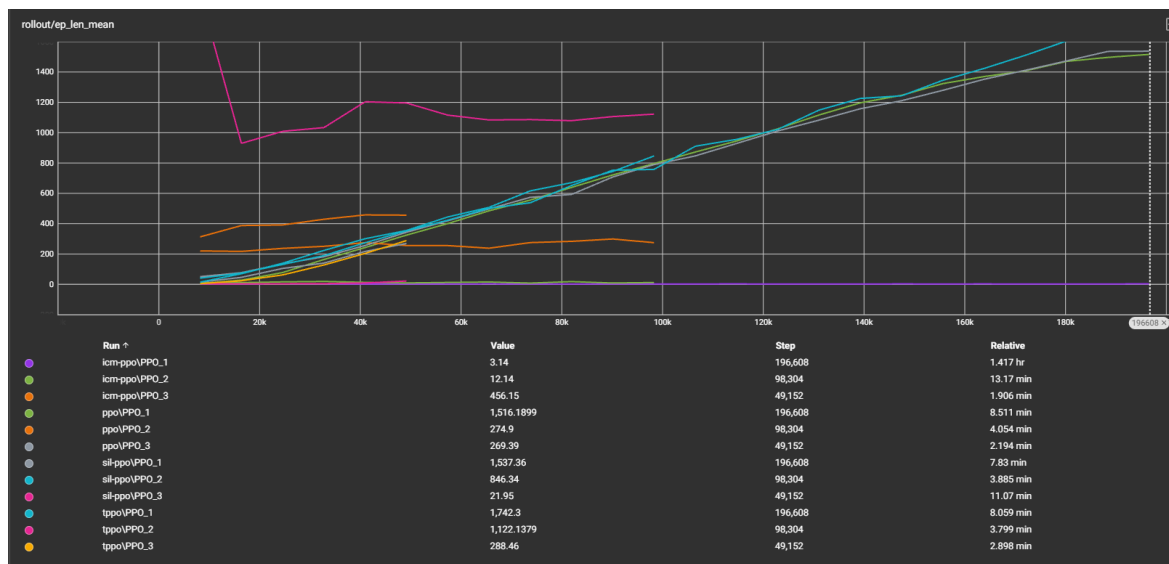


FIGURE 3.4: PPO episode lengths over training steps

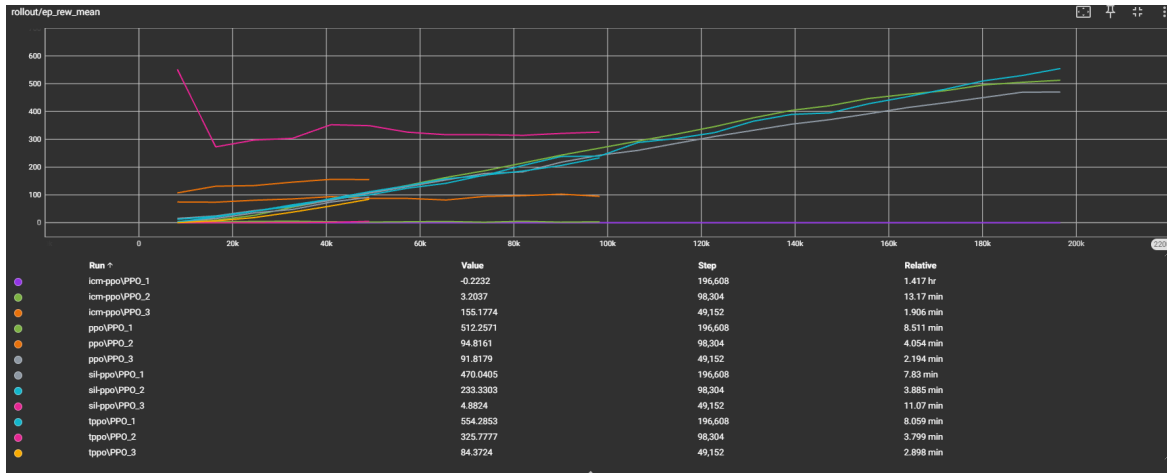


FIGURE 3.5: PPO cumulative rewards over training steps



FIGURE 3.6: PPO losses for rollout batches (which is why its smoother) over training steps



FIGURE 3.7: PPO other metrics as implemented by [6]

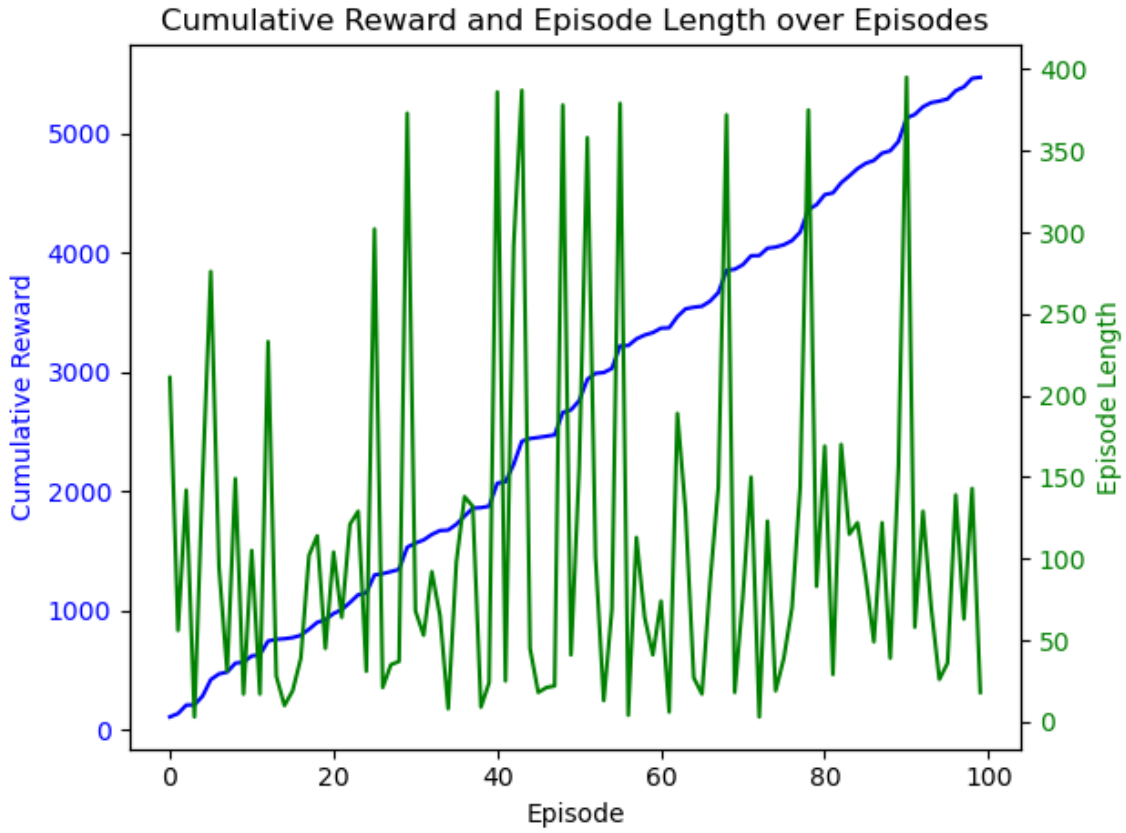


FIGURE 3.8: Overlaid DQN Metrics

3.3 Inference

Inference is among the most computationally intensive tasks in this setup. During inference, episode replays are collected, along with metrics such as cumulative rewards and episode lengths across multiple runs, to evaluate overall agent performance and stability.

3.3.1 Reward and Episode Lengths

The sparse action space presented challenges in implementing visualizations of action distributions. As a result, alternative metrics such as reward progression and episode lengths were prioritized for monitoring agent behavior.

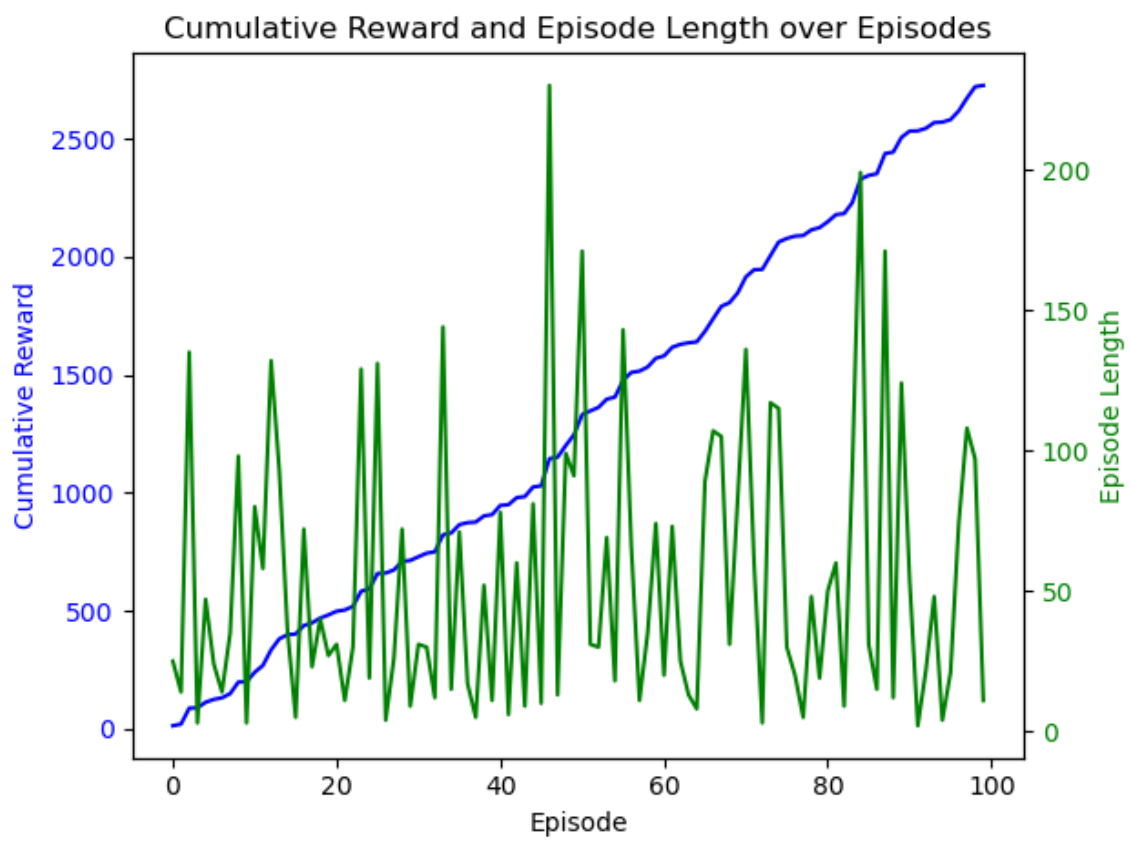


FIGURE 3.9: Overlaid 2DQN Metrics

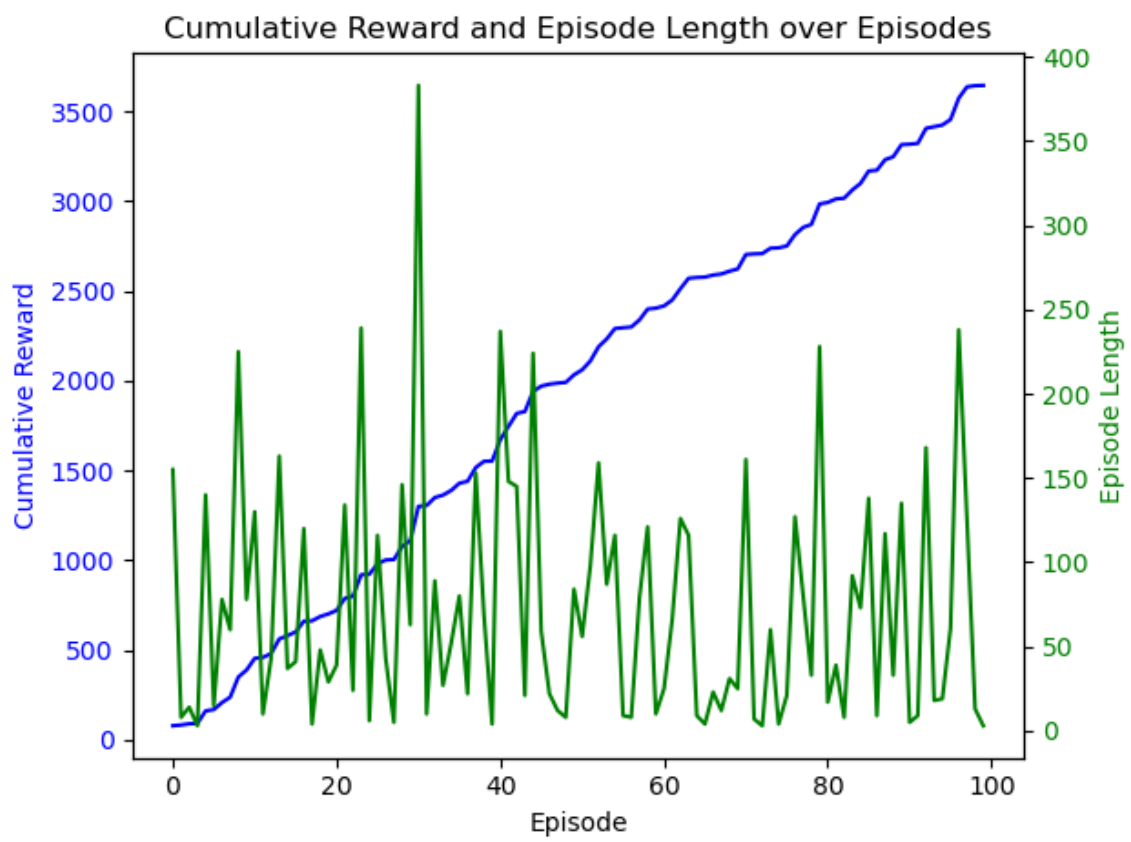


FIGURE 3.10: Overlaid DDQN Metrics

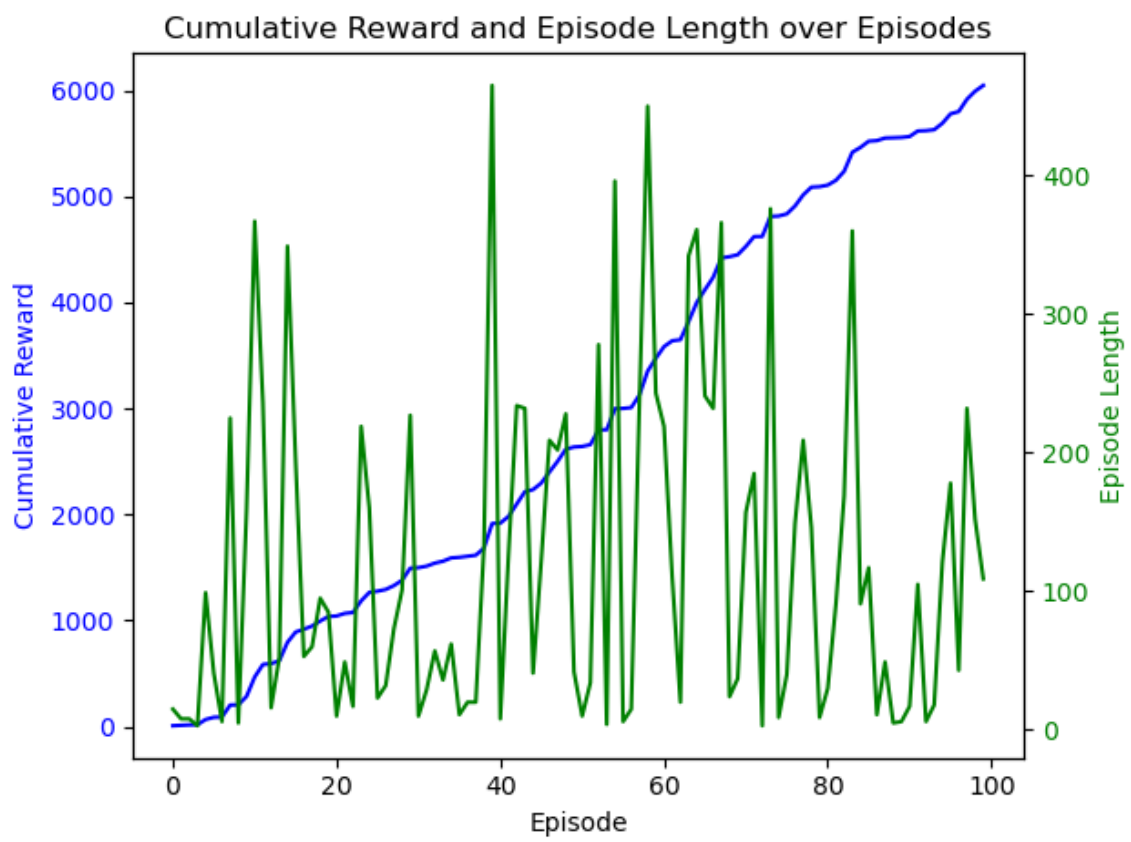


FIGURE 3.11: Overlaid PER-DQN Metrics

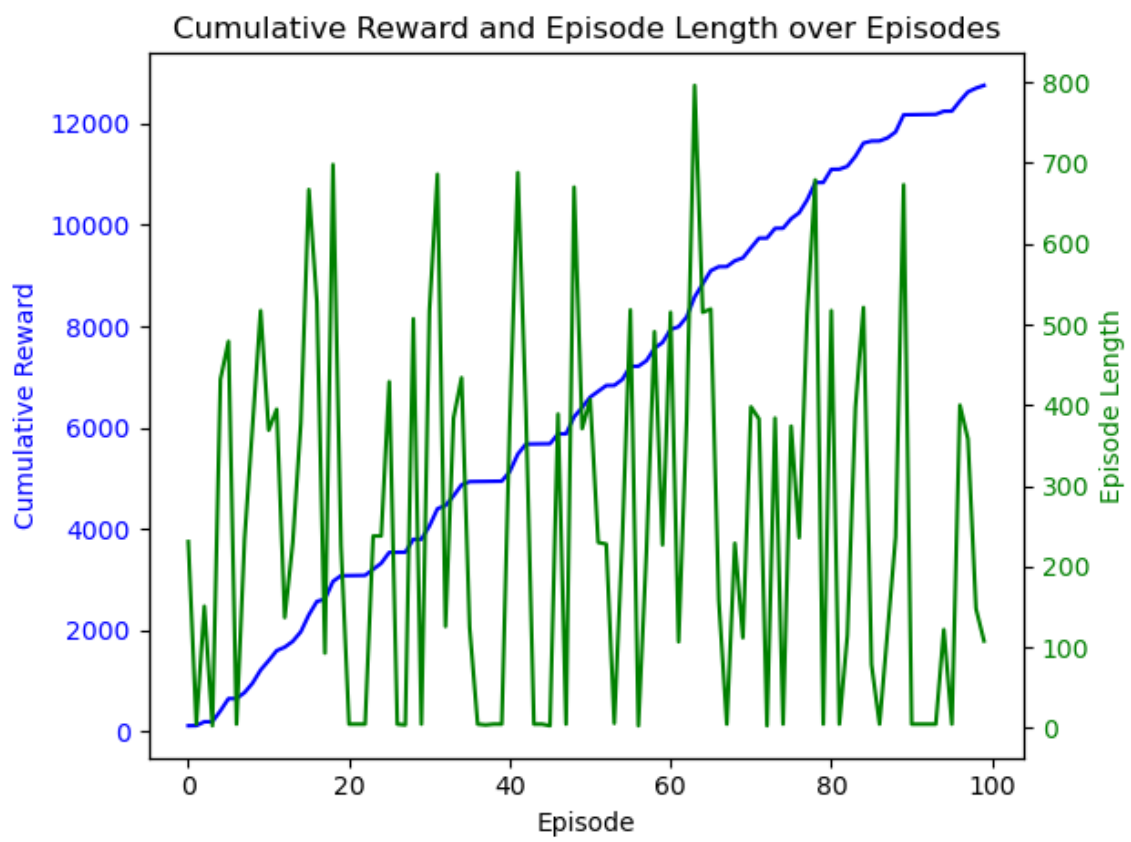


FIGURE 3.12: Overlaid PPO Metrics

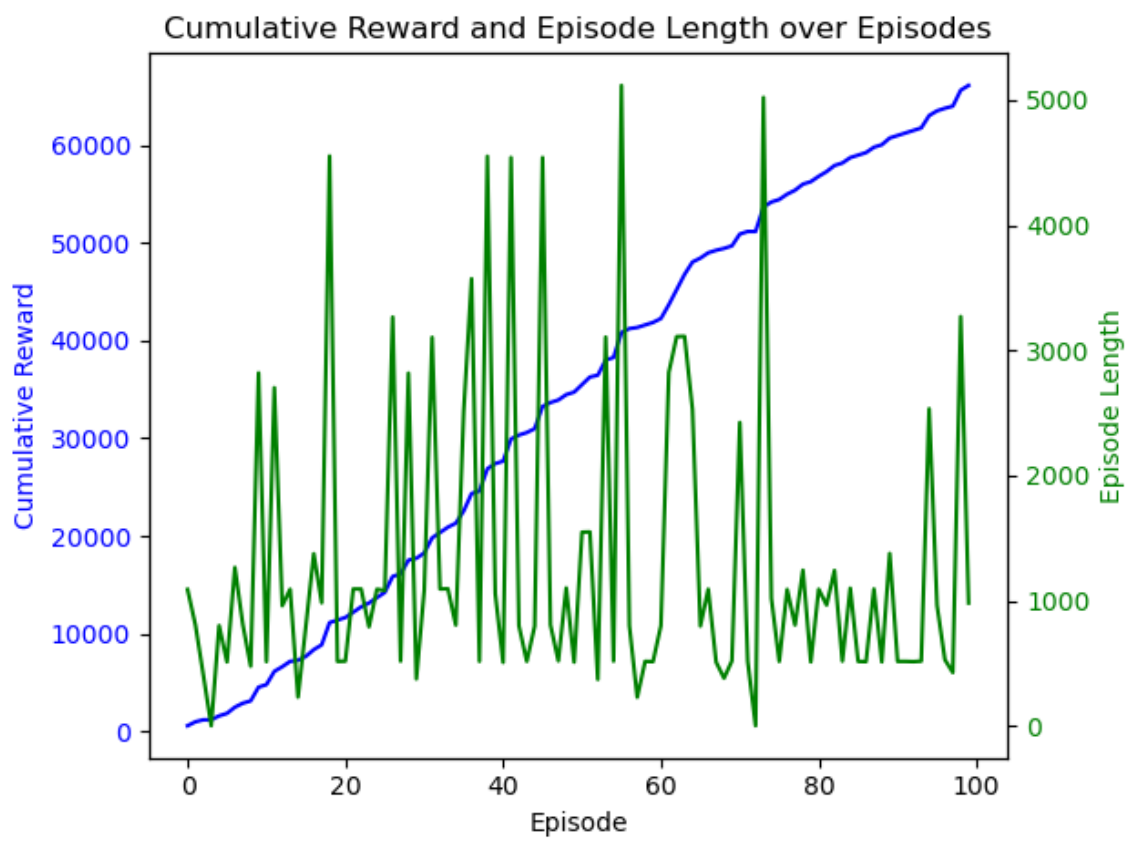


FIGURE 3.13: Overlaid TPPO Metrics

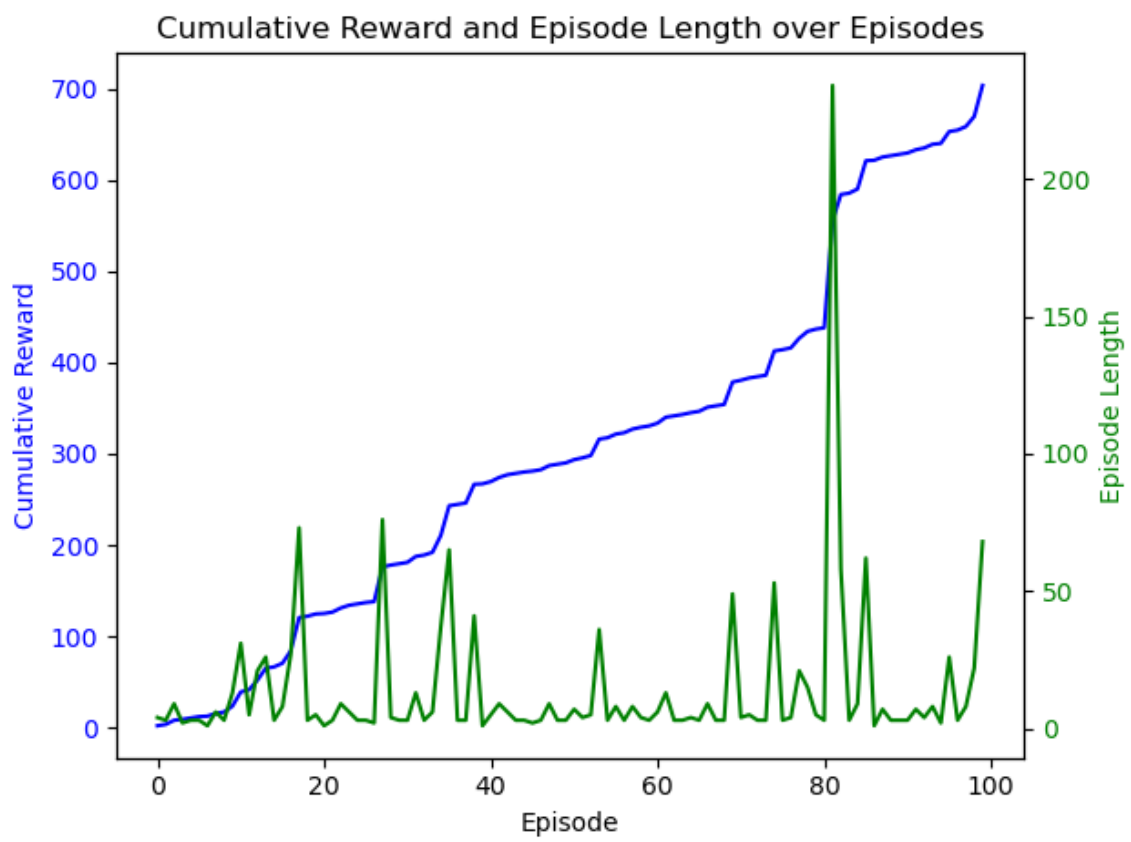


FIGURE 3.14: Overlaid ICM-PPO Metrics

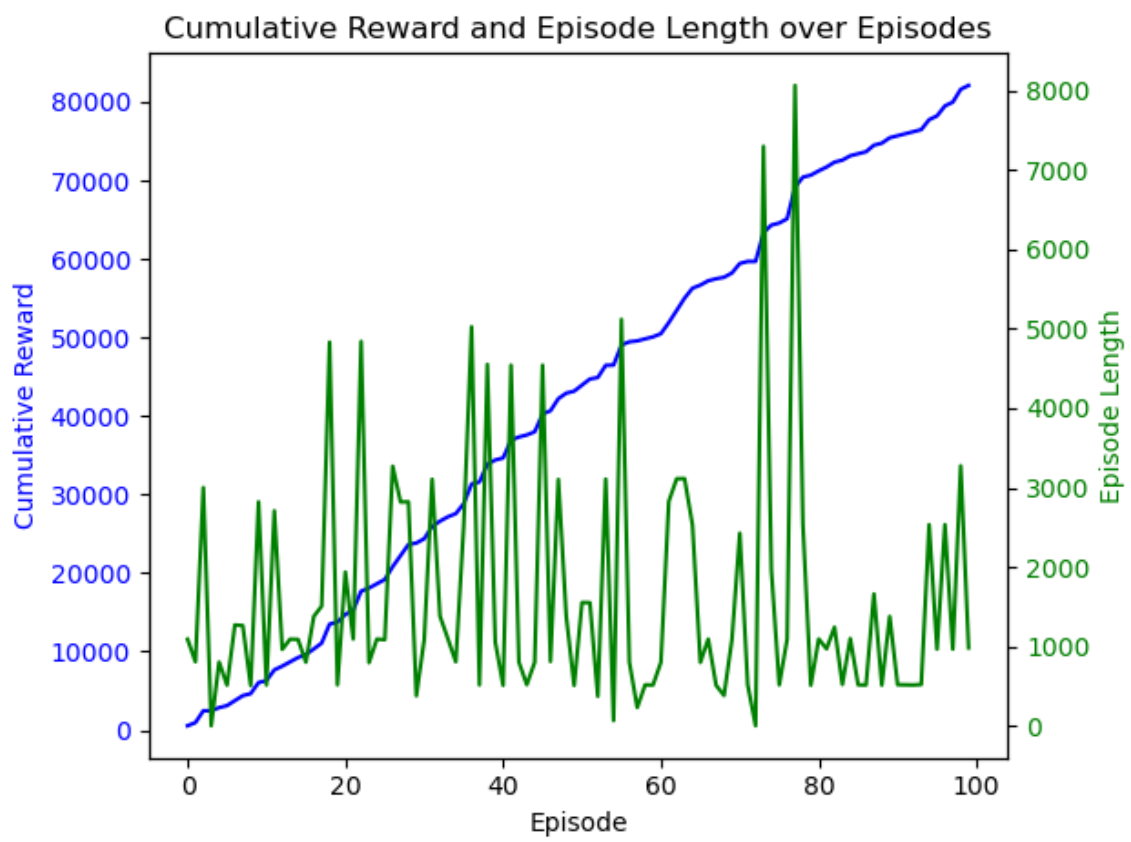


FIGURE 3.15: Overlaid SIL-PPO Metrics

[1] provides a built-in visualization tool, `grid2viz`, which was used to display and analyze episode replays. Additionally, an episode GIF is included in the submission, showcasing a well-performing model and episode (gifs are quite large, which is why only one is present).

3.4 Hyperparameters

A set of common hyperparameters was used, with specific hyperparameters tuned individually for each implementation. Fine-tuning was guided by prior performance results, and TensorBoard logs of these tuning runs are available in the GitHub repository for further inspection. Adjustments to batch size, learning rate, and discount factor had a significant impact on training performance. Beginning with a low learning rate facilitated early convergence, allowing the models to stabilize within a relatively small number of training steps

3.4.1 DQN Hyperparameters

- **learning_rate**: Starting at 1×10^{-5} , controlled by a scheduler function to adjust gradually.
- **buffer_size**: The maximum number of experiences in the replay buffer, set to 2500 to allow fast data turnover.
- **tau**: The soft update factor for target network updates, set to 0.005 to ensure stable transitions.
- **gamma**: The discount factor, set to 0.95, balancing immediate and future rewards.

3.4.2 PPO Hyperparameters

- **learning_rate**: Starting at 1×10^{-5} , modulated by a scheduler for gradual adjustment.
- **gamma**: Discount factor, set to 0.90, placing more emphasis on immediate rewards compared to DQN.

- **n_steps**: Number of steps before updating the policy, set to 2048, affecting update frequency.
- **batch_size**: Mini-batch size for each update, set to 1000, balancing memory efficiency and stable updates.
- **gae_lambda**: Lambda for Generalized Advantage Estimation, set to 0.95, providing smooth advantage estimates.
- **ent_coef**: Coefficient for entropy regularization, set to 0.01, encouraging exploration.
- **vf_coef**: Coefficient for value function loss, set to 0.5, balancing policy and value updates.
- **max_grad_norm**: Maximum gradient norm, set to 0.5, for gradient clipping to prevent large updates.
- **use_sde**: Set to False, indicating no use of Stochastic Differential Equations for noise control.
- **sde_sample_freq**: Set to -1, indicating no frequency adjustment for SDE-based exploration.

Bibliography

- [1] B. Donnot. *Grid2op- A testbed platform to model sequential decision making in power systems*. 2020. URL: [url{https://GitHub.com/Grid2Op/grid2op}](https://GitHub.com/Grid2Op/grid2op).
- [2] Ketan Doshi. *Reinforcement Learning Explained Visually (Part 5): Deep Q Networks, step-by-step — towardsdatascience.com*. <https://towardsdatascience.com/reinforcement-learning-explained-visually-part-5-deep-q-networks-step-by-step-5a5317197f4b>. [Accessed 25-10-2024].
- [3] Hado van Hasselt, Arthur Guez, and David Silver. *Deep Reinforcement Learning with Double Q-learning*. Dec. 8, 2015. arXiv: [1509.06461\[cs\]](https://arxiv.org/abs/1509.06461). URL: <http://arxiv.org/abs/1509.06461> (visited on 10/21/2024).
- [4] Amudhini P. Kalidas et al. “Deep Reinforcement Learning for Vision-Based Navigation of UAVs in Avoiding Stationary and Mobile Obstacles”. In: *Drones* 7.4 (Apr. 1, 2023), p. 245. ISSN: 2504-446X. DOI: [10.3390/drones7040245](https://doi.org/10.3390/drones7040245). URL: <https://www.mdpi.com/2504-446X/7/4/245> (visited on 10/21/2024).
- [5] Volodymyr Mnih et al. “Human-level control through deep reinforcement learning”. In: *Nature* 518.7540 (Feb. 1, 2015), pp. 529–533. ISSN: 1476-4687. DOI: [10.1038/nature14236](https://doi.org/10.1038/nature14236). URL: <https://doi.org/10.1038/nature14236>.
- [6] Antonin Raffin et al. “Stable-Baselines3: Reliable Reinforcement Learning Implementations”. In: *Journal of Machine Learning Research* 22.268 (2021), pp. 1–8. URL: <http://jmlr.org/papers/v22/20-1364.html>.
- [7] Tom Schaul et al. *Prioritized Experience Replay*. Feb. 25, 2016. arXiv: [1511.05952\[cs\]](https://arxiv.org/abs/1511.05952). URL: <http://arxiv.org/abs/1511.05952> (visited on 10/21/2024).
- [8] John Schulman et al. *Proximal Policy Optimization Algorithms*. Aug. 28, 2017. arXiv: [1707.06347\[cs\]](https://arxiv.org/abs/1707.06347). URL: <http://arxiv.org/abs/1707.06347> (visited on 10/21/2024).

- [9] Yuhui Wang et al. *Truly Proximal Policy Optimization*. Version Number: 2. 2019. DOI: [10 . 48550 / ARXIV . 1903 . 07940](https://doi.org/10.48550/ARXIV.1903.07940). URL: [https : // arxiv . org / abs / 1903 . 07940](https://arxiv.org/abs/1903.07940) (visited on 10/21/2024).
- [10] Ziyu Wang et al. *Dueling Network Architectures for Deep Reinforcement Learning*. Apr. 5, 2016. arXiv: [1511 . 06581 \[cs\]](https://arxiv.org/abs/1511.06581). URL: [http : // arxiv . org / abs / 1511 . 06581](http://arxiv.org/abs/1511.06581) (visited on 10/21/2024).